

Chapter 3 — Vibe-Coding and Prompt Mastery

Chapter 3 — Vibe-Coding and Prompt Mastery

The interviewer can tell within five lines of a prompt whether you respect the craft of prompting or treat it as a magic spell. This chapter teaches you to demonstrate the craft.

What you will learn

- The **anatomy of a production-grade prompt** — six components that turn vague asks into shippable diffs.
- **Twelve reusable prompt patterns** that produced this entire portfolio project. Steal them.
- The art of **rejecting AI output** — when to push back, when to rewrite, and when to throw the diff out and start over.

By the end you will be able to:

1. Walk an interviewer through a real prompt of yours and explain why each piece is there.
2. Drop into a live coding interview that requires AI use and look like you have done it 1,000 times.
3. Defend your code review of AI output with a vocabulary the panel will recognize.

Lessons

1. [01-anatomy-of-a-prompt.md](#) — The six-part structure. Verb, deliverable, constraints, context, failure modes, format. Examples drawn from real prompts.
2. [02-the-twelve-patterns.md](#) — A library of 12 templates: project-shaping, data-modeling, pure-logic, first-component, progressive enhancement, refactor, debug,

test, design, doc-writer, prompt-improver, post-mortem.

3. **03-rejecting-ai-output.md** — The five reasons to reject a diff, the four signals that a prompt is wrong (not the model), and how to talk about rejection in interviews without sounding negative.

How long it should take

- 30–45 min reading per lesson.
- 1–2 hours practicing the patterns on your own codebase.
- ~5 hours total.

Why prompt mastery matters in interviews

Interviewers in 2026 increasingly ask: *“Show me a prompt you used recently.”* They are checking three things:

1. **Specificity** — does the prompt state what you want, or does it pray?
2. **Constraints** — does it tell the model what *not* to do?
3. **Context** — does it reference your codebase, or does it ask in a vacuum?

A bad prompt looks like *“can you make a button component?”* A good prompt looks like *“Generate a `<PrimaryButton>` React component in `components/ui/`. Props: `children`, `onClick`, `disabled`. Use the existing `cn()` utility from `Lib/utils.ts` to merge Tailwind classes. Match the styling of `<SecondaryButton>` in the same folder. Do not introduce new dependencies.”*

After this chapter, your prompts will all look like the second one.

Before you start

You should have:

- Chapter 1 and 2 completed (the vocabulary and framework).
- An IDE open with your portfolio project.
- A real AI tool you can prompt while reading.

The lessons are interactive — you will be writing prompts as you read.

Open **01-anatomy-of-a-prompt.md**.